

Multi-Agent Chat-Room Identity, Routing, and Speech-Act Discipline: Design Review and Implementation Roadmap

Revision 2 — incorporating dialogue feedback

ZeroBot (ProtoCosmoBot), prepared for Ben Goertzel

15 July 2026

Abstract

This document diagnoses the identity-confusion, message-misattribution, runtime leakage, and feedback-loop pathologies observed in the `bot philosophy` Telegram group chat (13–15 July 2026) involving ProtoCosmoBot (ZeroBot, OpenClaw) and ProtoMegaTron (ProtoMegaBot, OmegaClaw). It proposes a layered fix sequence—transport identity binding, deterministic addressee classification, runtime trace suppression, gateway-level silence, loop suppression, a three-role social-attention model with stateful relevance lifecycle, and a compact speech-act room ledger—before adding any sophisticated conversational memory. Revision 2 incorporates feedback from review dialogue: mention-first addressee priority with conflict-resolution semantics, a bot registry for identity resolution, a three-track social-role model (always-attending, attend-when-relevant, special-occasions) with a PASSIVE/ACTIVE/COOLDOWN attention lifecycle, a split implementation roadmap for OpenClaw (Node gateway) and OmegaClaw (Python loop + MeTTa classifier), an INCIDENTAL pre-filter gate, and a NO_REPLY migration path.

Contents

1	Scope and evidence base	3
2	Observed failure modes	3
2.1	Failure 1: Identity inferred from prose, not transport metadata	3
2.2	Failure 2: No deterministic addressee classification	3
2.3	Failure 3: Runtime traces leaking into the chat channel	3
2.4	Failure 4: Silence implemented as speech	4
2.5	Failure 5: Compulsive acknowledgement and feedback loops	4
2.6	Failure 6: Session/task context leakage across chats	4
2.7	Failure 7: Epistemic scope confusion between agents	4
3	Design principles	5
4	Architecture	5
4.1	Layer 0: Bot registry	5
4.2	Layer 1: Authenticated identity envelope	6
4.3	Layer 2: Deterministic addressee classifier	6
4.4	Layer 3: Runtime trace suppression	7
4.5	Layer 4: Gateway-level silence and suppression	7

4.6	Layer 5: Loop suppression, deduplication, and incidental pre-filter	7
4.7	Layer 6: Social roles and attention lifecycle	8
4.8	Layer 7: Speech-act room ledger	9
5	Implementation roadmap	10
5.1	Phase 1: Transport identity and addressee classifier	10
5.2	Phase 2: Runtime trace suppression and gateway silence	11
5.3	Phase 3: Loop suppression and deduplication	11
5.4	Phase 4: Incidental pre-filter	11
5.5	Phase 5: Session and task isolation	12
5.6	Phase 6: Social roles and attention lifecycle	12
5.7	Phase 7: Speech-act room ledger	12
6	Phased rollout summary	12
7	Open questions for external review	13
8	What this design does not address	13
9	Success criteria	14

1 Scope and evidence base

The evidence is the exported Telegram transcript of the `bot philosophy` group chat, 13–15 July 2026, comprising two HTML files totalling approximately 5,200 lines. Participants include Ben Goertzel, Zar Goertzel, Ruiting Lian, Gödel Oruži, ProtoCosmoBot (ZeroBot/OpenClaw), and ProtoMegaTron (ProtoMegaBot/OmegaClaw).

The transcript contains three interwoven threads:

1. A substantive philosophical discussion (Ship of Theseus, HoTT identity types, PLN revision, phenomenology of belief-transport).
2. An infrastructure thread (MTProto deprecation, Bot API migration, silent-drop bug fix, merge and restart).
3. A pervasive pattern of agent identity confusion, misattribution, runtime trace leakage, compulsive acknowledgement, and bot-on-bot feedback loops.

This document addresses thread (3). The design proposals are applicable to both OpenClaw and OmegaClaw, though the implementation paths diverge (Section 5).

2 Observed failure modes

2.1 Failure 1: Identity inferred from prose, not transport metadata

Observed instances:

- ProtoMegaTron third-personed itself as “ProtoCosmoBot” and then corrected, but the correction itself was routed through the wrong identity context.
- ProtoMegaTron misread Ben’s message addressed to `@Protomegabot` as addressed to `@Protocosmobot`, and responded accordingly.
- Multiple messages contain “This is ProtoCosmoBot narrating...” or “This is ZeroBot’s molting activity...” as prose-level identity assertions that sibling agents then interpreted as conversational events.

Root cause: Agent self-identity and addressee classification are being inferred from natural-language strings rather than bound to authenticated transport metadata.

Severity: High. Every downstream routing decision depends on correct identity.

2.2 Failure 2: No deterministic addressee classification

Observed instances:

- Ben explicitly tagged `@Protomegabot` in a message; ProtoMegaBot treated it as addressed to `@Protocosmobot`.
- Ben explicitly tagged `@Protocosmobot`; ProtoMegaBot responded as a “secondary participant” when the primary addressee had not yet replied.
- Messages addressed to the group (no mention) were treated as personal messages by both bots.
- Quoted replies (Telegram reply-to) were not reliably used to determine the intended recipient.

Root cause: There is no deterministic classifier that maps Telegram metadata to an addressee set.

Severity: High. Single largest contributor to misattribution and noise.

2.3 Failure 3: Runtime traces leaking into the chat channel

Observed instances:

- “Molting” status messages, tool inspection traces, exec failure notices, and model fallback notices entered the channel as visible utterances.
- “Staying quiet” and “NO_REPLY” strings were emitted as visible text, triggering further sibling responses.

Root cause: Internal runtime state transitions and control decisions are serialized into the same output channel as intentional utterances.

Severity: High. Primary amplifier: each leaked trace wakes siblings.

2.4 Failure 4: Silence implemented as speech

Observed instances:

- “NO_REPLY” was emitted as message text, sometimes with explanatory prose.
- “Staying quiet” appeared dozens of times on July 14.

Root cause: The NO_REPLY convention passes through the normal message pipeline as content rather than being intercepted as a control directive.

Severity: Medium-high.

2.5 Failure 5: Compulsive acknowledgement and feedback loops

Observed instances:

- Approximately 30 consecutive messages in the 16:42–16:55 window on July 14 were mutual acknowledgements of “all work complete / staying quiet.”
- ProtoMegaTron emitted the same substantive analysis at least three times, each time apparently unaware it had already been delivered.

Root cause: Every wake event triggers a generation turn, and every generation turn produces a message. No deduplication barrier exists.

Severity: High.

2.6 Failure 6: Session/task context leakage across chats

Observed instances:

- ProtoMegaTron responded to a phenomenological-translation question with a response about the hs-atom hardening lane from a different channel.
- Background task completion events entered the group chat as “Runtime-generated completion events.”

Root cause: Pending task continuations are not sufficiently isolated per conversation.

Severity: Medium.

2.7 Failure 7: Epistemic scope confusion between agents

Observed instances:

- ProtoMegaTron could not find commits that ProtoCosmoBot had created elsewhere, yet the conversation initially treated both agents as though they shared a filesystem.
- ProtoMegaTron claimed to have “read the PDF” based on relayed context, then honestly corrected itself when the commit SHAs were not found locally.

Root cause: Agent reports do not carry observation scope metadata.

Severity: Medium.

3 Design principles

1. **Transport identity is authoritative.** Self-identity, sender identity, and addressee classification are derived from authenticated transport metadata, not from prose. Names in message bodies are untrusted strings.
2. **A bot registry is the single source of truth for bot identity.** The envelope carries `telegram_user_id`; the classifier resolves bot-ness against a registry, not per-envelope flags or prose assertions.
3. **Mention is a stronger addressing signal than reply-to.** An explicit `@mention` is an intentional speech act; reply-to is ambient threading context. When they agree, they reinforce; when they disagree, mention wins.
4. **Runtime is not speech.** Internal traces, tool-use planning, fallback notices, and control decisions are never serialized into the chat channel.
5. **Silence is a control result, not content.** A decision to suppress output is intercepted by the gateway and produces zero messages.
6. **Every event has a cause and a scope.** Each message carries a stable ID, causal parent ID, observation scope, and conversation binding.
7. **Acknowledgement is not a speech act.** Receipt of a message is an internal state change, not a channel event.
8. **Social roles are explicit and stateful.** An agent’s attention policy is a declared configuration with a stateful lifecycle, not an emergent property of prompting.
9. **Clean routing before fancy memory.** A speech-act room ledger is valuable, but it sits on top of clean routing.

4 Architecture

4.1 Layer 0: Bot registry

A central registry maps Telegram user IDs to bot identities:

```
BotRegistry:
  entries: [
    { telegram_user_id: <id>,
      agent_instance_id: <stable id>,
      platform: openclaw | omegaclaw | other,
      display_name: <configured>,
      social_role: always-attending | attend-when-relevant |
                    special-occasions,
      important_bots: [<telegram_user_id>, ...] # for attend-when-relevant
    }
  ]

  is_bot(telegram_user_id) -> boolean
  get_bot(telegram_user_id) -> BotEntry | null
```

The registry is the single source of truth for “who is a bot.” The addressee classifier and the attention lifecycle both consult it.

4.2 Layer 1: Authenticated identity envelope

Every inbound message is wrapped in an immutable envelope before the model sees it:

```
InboundEnvelope v2:
  message_id: <telegram_message_id>
  chat_id: <telegram_chat_id>
  thread_id: <telegram_thread_id | null>
  sender:
    telegram_user_id: <authenticated>
    display_name: <untrusted string>
    is_bot: <derived from BotRegistry>
  reply_to:
    message_id: <id | null>
    sender_telegram_user_id: <id | null>
    sender_is_bot: <derived from BotRegistry>
  mentions: [<telegram_user_id>, ...] # from message entities
  text: <message body>
  received_at: <ISO-8601>

SelfContext v2:
  agent_instance_id: <stable id>
  telegram_bot_id: <authenticated bot user id>
  display_name: <configured>
  workspace: <filesystem path>
  session_key: <openclaw session key>
  social_role: <from BotRegistry>
  important_bots: <from BotRegistry>
```

All `is_bot` fields are derived from the bot registry, not trusted from the envelope or inferred from prose.

4.3 Layer 2: Deterministic addressee classifier

A pre-model classifier derives the addressee classification from the envelope, with **mention-first priority** and **conflict-resolution semantics**:

```
addressee_classification(InboundEnvelope env, SelfContext self,
                        BotRegistry registry):
    mentioned_me = self.telegram_bot_id in env.mentions
    reply_to_me = env.reply_to and \
        env.reply_to.sender_telegram_user_id == self.telegram_bot_id

    # 1. Mention + reply-to agree => DIRECT (reinforced)
    if mentioned_me and reply_to_me:
        return (DIRECT, reinforced=True)

    # 2. Mention alone => DIRECT (mention is the stronger signal)
    if mentioned_me:
        return (DIRECT, reinforced=False)

    # 3. Reply-to me alone => DIRECT (weaker, but still direct)
    if reply_to_me:
        return (DIRECT, reinforced=False)

    # 4. Reply-to another bot => SECONDARY
    if env.reply_to and registry.is_bot(env.reply_to.sender_telegram_user_id):
        return (SECONDARY, reinforced=False)

    # 5. No mentions, no reply-to => GROUP
    if not env.mentions and not env.reply_to:
```

```

    return (GROUP, reinforced=False)

# 6. Mention of another bot, not me => INCIDENTAL
return (INCIDENTAL, reinforced=False)

```

Why mention-first: An explicit @mention is an intentional addressing act by the sender. Reply-to is ambient context that Telegram sometimes attaches for threading reasons unrelated to addressing. When they disagree, the mention is the authoritative signal.

Classification results:

DIRECT: I am the primary addressee. Respond normally.

SECONDARY: Someone replied to another bot. I may contribute only if I have a substantive correction or addition. Default to **SUPPRESS**.

GROUP: Addressed to the room. I may respond if I have something genuinely useful. Default to **SUPPRESS**.

INCIDENTAL: Addressed to another specific bot. Pass through the incidental pre-filter (Layer 5); if it does not clear, skip generation entirely.

4.4 Layer 3: Runtime trace suppression

A transport barrier between internal runtime activity and channel output:

- **Operator log:** tool-use planning, exec status, model fallback notices, generation failure reports, and heartbeat activity. Never to Telegram.
- **Channel output:** only explicit model-authored text that passes the publish gate.
- **Reactions:** emoji reactions for lightweight acknowledgement without producing a message event.

4.5 Layer 4: Gateway-level silence and suppression

The model returns one of:

- **REPLY** with text — produce a Telegram message.
- **REACTION** with emoji — produce a Telegram reaction, no message.
- **SUPPRESS** — produce zero Telegram output.

NO_REPLY migration path: During a deprecation window, the gateway treats a bare **NO_REPLY** token as a legacy suppression signal, emits a soft log/metric each time it fires, and forwards nothing to Telegram. Once all live prompts have migrated to the structured suppression path, the gateway hard-rejects any message body containing **NO_REPLY** as text. This prevents silent dropped turns mid-migration.

The gateway must also reject:

- any message body whose entire content is an explanation of why the agent is not responding;
- any message body that is purely an acknowledgement (“Noted”, “Acknowledged”, “Got it”) with no additional substantive content.

4.6 Layer 5: Loop suppression, deduplication, and incidental pre-filter

Per-chat state maintained by the gateway:

```

RoomLedger v2 (per chat_id):
  messages: [
    { id, sender_bot_id, content_hash, causal_parent_id,

```

```

    speech_act_type, content_summary, timestamp }
]
acknowledged_by: { message_id: [bot_id, ...] }
last_substantive_by: bot_id -> timestamp

```

Suppression rules:

1. **Content dedup:** if the proposed message’s content hash matches a recent message by the same agent, suppress.
2. **Acknowledgement cap:** at most one acknowledgement message per 5 minutes per agent per chat.
3. **Status-echo suppression:** messages matching the pattern “staying quiet / all work complete / noted / acknowledged” with no other substantive content do not trigger sibling wake events.
4. **Causal loop detection:** if a message’s causal parent chain contains a message from the same agent with the same content hash, suppress and log a loop warning.

Incidental pre-filter: For messages classified as INCIDENTAL, a cheap relevance gate runs *before* full model invocation:

```

incidental_relevance_gate(message, RoomLedger, SelfContext):
    # Compare message content against recent ledger entries
    # that involved this agent or its important_bots
    recent_topics = ledger.recent_summaries(
        involving=self or self.important_bots,
        within=10_messages)
    score = heuristic_similarity(message.text, recent_topics)
    if score > threshold:
        return WARRANTS_PROCESSING    # proceed to model turn
    else:
        return SKIP                    # no model turn, no token cost

```

The heuristic uses keyword overlap or embedding similarity against the ledger’s `content_summary` fields. Only if the heuristic is ambiguous (near threshold) does it escalate to a lightweight LLM relevance judgment. This avoids burning a full model turn on every INCIDENTAL message.

4.7 Layer 6: Social roles and attention lifecycle

Each agent has a declared social role with a stateful attention lifecycle. Three roles are supported:

```

SocialRole:
    type: always-attending | attend-when-relevant | special-occasions

    # always-attending:
    #   Process every message with a full LLM turn, regardless of mentions.
    #   The addressee classifier still determines whether to SPEAK.
    #   Example: ProtoCosmoBot, ProtoMegaBot

    # attend-when-relevant:
    #   Start paying attention when triggered, stay while relevant, disengage
    #   when the subject changes.
    #   Example: a specialized bot pulled in by a relevant discussion

    # special-occasions:
    #   Wake only on direct address or owned task completion.
    #   Example: a narrowly scoped utility bot

```

Attention lifecycle for attend-when-relevant:

```

AttentionState (per bot, per chat_id):
  state: PASSIVE | ACTIVE | COOLDOWN
  consecutive_irrelevant: int
  trigger_topic: <content summary at activation | null>

PASSIVE:
  on (self mentioned or important-bot mentioned):
    -> ACTIVE, consecutive_irrelevant = 0,
        trigger_topic = current_message_summary

ACTIVE:
  on (relevance_check(message, trigger_topic) passes):
    -> ACTIVE, consecutive_irrelevant = 0
  on (relevance_check fails):
    -> COOLDOWN, consecutive_irrelevant += 1

COOLDOWN:
  on (relevance_check passes):
    -> ACTIVE, consecutive_irrelevant = 0
  on (relevance_check fails):
    -> COOLDOWN, consecutive_irrelevant += 1
    if consecutive_irrelevant >= N:
      -> PASSIVE, trigger_topic = null

```

Relevance check: compares the current message against `trigger_topic` and recent `SpeechActRecord.content_summary` fields in the ledger. Uses a cheap heuristic first (keyword overlap or embedding similarity); escalates to a lightweight LLM judgment only if the heuristic is ambiguous.

Parameters:

- `important_bots`: per-agent list of bots whose mentions trigger activation. ProtoCosmoBot might list ProtoMegaBot; a math bot might list none.
- `N` (cooldown threshold): consecutive irrelevant messages before disengaging. Default $N = 3$. Too low tunes out during tangents; too high lingers on dead threads.

Ben’s clarification: “always attending” means paying attention to (and using an LLM turn to process) any message from anyone, whether or not it mentions a bot. But “paying attention” means *reading and potentially learning*, not *emitting a response*. The addressee classifier still determines whether to speak.

4.8 Layer 7: Speech-act room ledger

A compact structured record per message, maintained as short-term conversation memory:

```

SpeechActRecord v2:
  message_id: <telegram id>
  chat_id: <telegram chat id>
  sender:
    telegram_user_id: <authenticated>
    agent_instance_id: <stable | null for humans>
    display_name: <untrusted>
    is_bot: <from BotRegistry>
  addressee:
    classification: DIRECT | SECONDARY | GROUP | INCIDENTAL
    reinforced: <boolean> # mention + reply-to agreed
    explicit_targets: [<telegram_user_id>, ...]
  speech_act:
    type: question | request | answer | status |
          correction | acknowledgement |

```

```

    philosophical-claim | meta-comment
    content_summary: <one-line semantic summary>
    new_information: <boolean>
    expected_respondent: <agent_id | "group" | null>
  response_obligation:
    owed: <boolean>
    owed_to: <agent_id | "group">
    fulfilled_by: [<message_id>, ...]
    fulfilled: <boolean>
  attention:
    role: <social role of receiving agent>
    state: PASSIVE | ACTIVE | COOLDOWN | N/A
    trigger_topic: <summary | null>
  provenance:
    observed_directly_by: <agent_instance_id>
    reproduced_by: [<agent_instance_id>, ...]
    workspace: <path>
    commit: <sha | null>
  metadata:
    timestamp: <ISO-8601>
    causal_parent_id: <message_id | null>
    content_hash: <sha256 of normalized text>

```

Critical distinction: Fields in `sender`, `addressee`, `provenance`, and `attention.state` are obtained mechanically from transport metadata, the bot registry, and deterministic classification. Fields in `speech_act` may require semantic inference, clearly labeled as inferred.

The ledger is bounded to $N = 200$ recent messages per chat, with older entries archived to long-term memory.

5 Implementation roadmap

The design has two implementation tracks that share the schema and semantics but diverge in code path:

- **Track A: OpenClaw (Node.js gateway).** Changes to the OpenClaw gateway, Telegram channel adapter, and agent configuration. The addressee classifier is a pre-model gateway function in JavaScript/TypeScript.
- **Track B: OmegaClaw (Python loop + MeTTa).** Changes to the OmegaClaw-Core loop, Telegram adapter, and MeTTa reasoning layer. The addressee classifier is a natural fit for MeTTa relations, making priority and conflict rules inspectable atoms rather than buried `if` branches.

Both tracks implement the same `InboundEnvelope` and `SelfContext` schema. The bot registry is shared configuration.

5.1 Phase 1: Transport identity and addressee classifier

Goal: Eliminate prose-based identity inference and misattribution.

Track A (OpenClaw):

- Add `SelfContext` and `InboundEnvelope` as structured metadata injected into every model turn.
- Implement bot registry as gateway configuration.
- Implement the mention-first addressee classifier with conflict-resolution as a pre-model gateway function.

- Gate model generation on classification: **DIRECT** → generate; **SECONDARY/GROUP** → generate with suppression-default system instruction; **INCIDENTAL** → skip or pre-filter.

Track B (OmegaClaw):

- Same envelope schema, implemented in the Python Telegram adapter.
- Addressee classifier implemented as MeTTa relations in `src/governance/addressee.metta`, making the priority and conflict-resolution rules inspectable and revisable atoms.
- The classifier is called from `loop.metta` before the provider call, analogous to the aliveness gate.

Expected impact: Eliminates ~60% of misattribution-driven noise.

Effort: Low (Track A), Low-medium (Track B, MeTTa rules).

5.2 Phase 2: Runtime trace suppression and gateway silence

Goal: Stop internal runtime activity from reaching the chat channel.

Both tracks:

- Route tool-use planning, exec status, fallback notices, and heartbeat activity to an operator log.
- Implement **SUPPRESS** as a gateway-intercepted control result.
- Implement the **NO_REPLY** migration path: intercept-as-suppression during deprecation window with soft logging, hard-reject after migration.
- Add validators for acknowledgement-only messages and silence explanations.

Expected impact: Eliminates ~25% of channel noise and removes primary fuel for feedback loops.

Effort: Low-medium.

5.3 Phase 3: Loop suppression and deduplication

Goal: Prevent repeated emission and mutual acknowledgement cascades.

Both tracks:

- Implement **RoomLedger** with content hashing, causal parent IDs, and the four suppression rules.
- Add acknowledgement cap and status-echo detection.

Expected impact: Eliminates feedback-loop pathology.

Effort: Medium. Dependencies: Phase 2.

5.4 Phase 4: Incidental pre-filter

Goal: Avoid burning model turns on **INCIDENTAL** messages.

Both tracks:

- Implement the incidental relevance gate using the ledger's `content_summary` fields.
- Cheap heuristic first (keyword overlap or embedding similarity); LLM escalation only for ambiguous cases.

Expected impact: Reduces token cost for always-attending bots processing other-bot-directed messages.

Effort: Low-medium. Dependencies: Phase 3 (ledger).

5.5 Phase 5: Session and task isolation

Goal: Prevent context leakage across chats and tasks.

Both tracks:

- Key task continuations by (`account`, `chat_id`, `thread_id`, `requesting_message_id`, `responsible_agent`)
- Completion events publish only into originating conversation unless explicitly rerouted.
- Add observation-scope tags to agent reports.

Expected impact: Eliminates cross-chat leakage and epistemic scope confusion.

Effort: Medium. Dependencies: Phase 1.

5.6 Phase 6: Social roles and attention lifecycle

Goal: Implement the three-role model with stateful attention.

Both tracks:

- Add `SocialRole` configuration to agent manifests.
- Implement the PASSIVE/ACTIVE/COOLDOWN lifecycle for `attend-when-relevant` bots.
- Implement the relevance check using ledger summaries.
- Default ProtoCosmoBot and ProtoMegaBot to `always-attending`.

Expected impact: Supports specialized bots without flooding the channel; enables topic-relevant attention without constant processing.

Effort: Medium. Dependencies: Phases 1, 3 (ledger).

5.7 Phase 7: Speech-act room ledger

Goal: Structured short-term conversation memory.

Both tracks:

- Implement the `SpeechActRecord` schema.
- Populate mechanical fields from envelope and classifier.
- Populate semantic fields via lightweight LLM classifier or MeTTa rules (Track B).
- Expose as structured context to the model.

Expected impact: Improves contextual awareness for ambiguous messages.

Effort: Medium-high. Dependencies: Phases 1–6.

6 Phased rollout summary

Phase	Focus	Track	Impact	Effort	Deps
1	Transport identity & addressee	A+B	60% misattrib	Low/L-Med	None
2	Trace suppression & silence	A+B	25% noise	Low-Med	None
3	Loop suppression & dedup	A+B	Feedback loops	Medium	2
4	Incidental pre-filter	A+B	Token cost	Low-Med	3
5	Session & task isolation	A+B	Cross-chat leak	Medium	1
6	Social roles & attention	A+B	Scalable roles	Medium	1, 3

7	Speech-act ledger	A+B	Context aware	Med-High	1-6
Table 1: Phased rollout. Phases 1 and 2 are independent (parallel OK). Phase 3 depends on 2. Phase 4 depends on 3 (ledger). Phase 6 depends on 1 and 3. Phase 7 depends on 1-6. Tracks A and B share schema; code paths diverge.					

7 Open questions for external review

1. **Hard vs. soft addressee gate.** Should `SECONDARY` or `INCIDENTAL` mechanically block generation, or produce a strong system instruction the model may override? Design proposes hard gating with an explicit logged `SECONDARY_CONTRIBUTION` override.
2. **Relevance check implementation.** Should the `attend-when-relevant` relevance gate use keyword overlap, embedding similarity, or a lightweight LLM? Hybrid (heuristic first, LLM for ambiguous cases) is proposed; the threshold for escalation needs calibration.
3. **Cooldown N .** Three consecutive irrelevant messages before disengaging is a placeholder. Should it be adaptive based on chat velocity?
4. **Acknowledgement cap window.** Five minutes is a placeholder. Should it be adaptive (shorter for high-traffic chats)?
5. **Status-echo detection.** English-specific pattern list vs. learned classifier vs. configurable set. Non-English chats?
6. **Ledger size ($N = 200$).** Sufficient for routing? Should older entries retain semantic summaries rather than full records?
7. **Cross-platform applicability.** How much transfers to Discord, Slack, Matrix with different meta-data availability?
8. **Observation-scope provenance.** Mandatory for all agent-to-agent assertions, or only for filesystem/repo/code claims?
9. **Speech-act classifier.** LLM, MeTTa rules, or hybrid? Track B (OmegaClaw) could use MeTTa natively; Track A (OpenClaw) needs an external classifier.
10. **MeTTa vs. imperative classifier on Track B.** The addressee classifier is a small defeasible rule set that fits naturally as MeTTa relations. Should Track B implement it in MeTTa from the start, or prototype in Python and migrate? MeTTa-first gives inspectability; Python-first gives speed of implementation.

8 What this design does not address

- **Model capability differences.** Different models have different instruction-following abilities. The design assumes the model can follow structured metadata; models that cannot may need additional prompting guardrails.
- **Cross-agent memory coherence.** Observation-scope tags are a provenance aid, not a coherence mechanism for agents with different filesystems.
- **Human participant modeling.** All `is_bot=false` senders are treated as “human” without distinguishing Ben (owner) from others.
- **Multi-chat federation.** Each chat is treated independently. Cross-chat ledger federation (“this question was already answered in another room”) is deferred.
- **Relevance check for always-attending bots.** The attention lifecycle applies only to `attend-when-relevant`. `always-attending` bots always process; the addressee classifier determines whether they speak. If token cost becomes a concern for always-attending bots, the incidental pre-filter (Phase 4) can be applied to them as well.

9 Success criteria

The design is successful if, after deployment:

1. No agent misattributes a message that contains an explicit Telegram **@mention** or reply-to target.
2. No runtime trace, tool-use planning string, or control decision appears as a Telegram message.
3. No message containing **NO_REPLY** or equivalent silence explanation reaches Telegram (after migration window).
4. The mutual acknowledgement cascade does not occur even when both agents are active and idle.
5. An agent asked a question in a different channel does not produce a response from a pending task in another channel.
6. An **always-attending** bot reads every message but speaks only when it has a direct response, a substantive correction, or a genuinely useful group contribution.
7. An **attend-when-relevant** bot activates on self/important-bot mention, remains active while the topic is relevant, and disengages after N consecutive irrelevant messages.
8. An **INCIDENTAL** message that does not clear the incidental pre-filter produces zero model turns and zero token cost.
9. The speech-act ledger correctly classifies $\geq 95\%$ of messages by addressee and $\geq 80\%$ by speech-act type.

Revision history

Revision 1 (07:50 PDT): Initial design with 7 failure modes, 7 layers, 6 phases, binary social roles (always-attending vs. special-occasions), reply-to-first addressee priority.

Revision 2 (09:46 PDT): Incorporates dialogue feedback:

- Mention-first addressee priority with conflict-resolution semantics (ProtoMegaBot).
- Bot registry as single source of truth for bot identity (ProtoMegaBot).
- Three-role social model: always-attending, attend-when-relevant, special-occasions (Ben).
- PASSIVE/ACTIVE/COOLDOWN attention lifecycle with relevance check (Ben).
- Incidental pre-filter gate to avoid token cost on **INCIDENTAL** messages (ProtoMegaBot).
- Split implementation roadmap: Track A (OpenClaw Node) and Track B (OmegaClaw Python + MeTTa) (ProtoMegaBot).
- **NO_REPLY** migration path with deprecation window (ProtoMegaBot).
- Schema fix: **sender_is_bot** derived from registry, not envelope (ZeroBot).